

UNIVERSIDAD DE MANIZALES

Universidad de Manizales
Tecnología en Desarrollo de Software para Negocios Digitales
Asignatura: Seguridad en Aplicaciones

Evaluación de Compra Segura frente a ISO 27002
Informe técnico de auditoría: evaluación de controles ISO/IEC 27002 y hallazgo de autenticidad en el sistema Compra Segura

Integrantes: Pablo Nieto Pérez (trabajo individual)

Docente: Sergio Pinilla Valencia

Manizales, 12 de Septiembre de 2026

Tabla de contenido

Tabla de contenido 2
Tabla de figuras 2
4. Introducción 3
5. Objetivos 4
5.1 Objetivo general 4
5.2 Objetivos específicos 4
6. Justificación 4
7. Cuerpo principal / Desarrollo 5
7.1 Contexto y metodología de evaluación 5
7.2 Evaluación de los dominios ISO 27002 5
7.2.1 A.9 Control de acceso 6
7.2.2 A.10 Criptografía 8
7.2.3 A.12 Seguridad de las operaciones 9
7.2.4 A.14 Adquisición, desarrollo y mantenimiento de sistemas 10
7.2.5 A.16 Gestión de incidentes de seguridad de la información 11
7.2.6 Resumen del checklist 12
7.3 Evidencias técnicas 15
7.4 Hallazgo de autenticidad 16
7.4.1 Situación observada 17
7.4.2 ¿Por qué compromete la autenticidad y no otro principio? 17
7.5 Discusión final 18
7.6 Reto del «Auditor Externo» 19
8. Discusión general 20
9. Conclusiones 21
10. Bibliografía 22
11. Anexos 23
Anexo A. Interacción con herramientas de inteligencia artificial 23

Tabla de figuras

Figura 1. Repositorio del proyecto.
Figura 2. Ausencia de index/enrutamiento y de caracteres mínimos en registro.php.
Figura 3. El sistema no valida la longitud mínima de la contraseña.
Figura 4. Tabla usuarios: solo usuario y clave.

Figura 5. Conexión directa a la base de datos y ausencia de .env.
Figura 6. Hallazgo de autenticidad en el formulario de acceso.

4. Introducción

El propósito de este informe es dar continuidad a la auditoría de la clase 2, que fue en donde identificamos y clasificamos hallazgos de seguridad sobre la aplicación Compra Segura. En esta tercera actividad ya no se trata de encontrar fallas, sino de medir el nivel de cumplimiento de la app frente a cinco dominios de control del estándar ISO/IEC 27002, y vamos aportando, para cada uno, evidencia concreta y que sea verificable en el código fuente o en la base de datos (ver Figura 1).

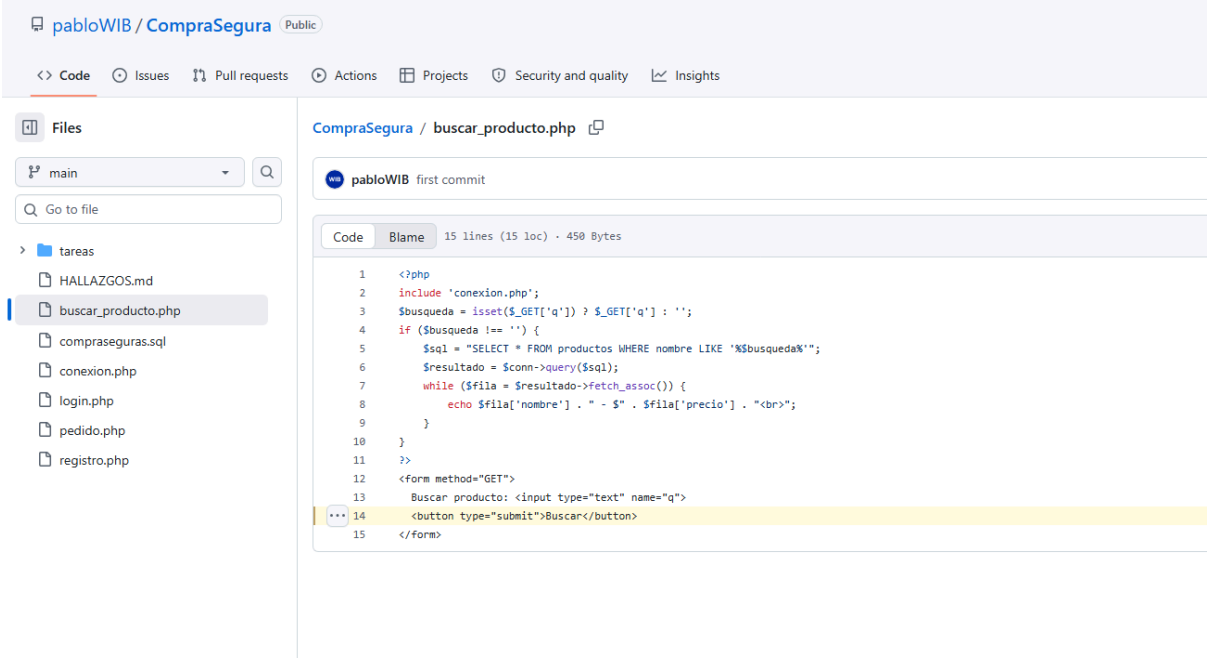


Figura 1. En esta imagen vemos el repo que tenemos para los avances del proyecto.

5. Objetivos

5.1 Objetivo general

El objetivo es documentar y justificar un hallazgo específico sobre el principio de autenticidad en el formulario de inicio de sesión, y responder a las dos preguntas de la discusión final, y emitir una respuesta a la objeción planteada.

5.2 Objetivos específicos

Para llegar a ese objetivo general nos pusimos unos objetivos más puntuales, y cada uno se puede ver después en una parte del desarrollo y en una de las conclusiones:
Objetivo específico 1: Evaluar los cinco dominios de la ISO/IEC 27002 (A.9, A.10, A.12, A.14 y A.16) sobre Compra Segura, dejando para cada control el estado y la evidencia que se puede verificar en el código o en phpMyAdmin (ver 7.2 y 7.3).
Objetivo específico 2: Documentar el hallazgo de autenticidad del login.php con una prueba que cualquiera pueda repetir (ver 7.4).
Objetivo específico 3: Priorizar cuál dominio se tiene que corregir primero, según lo que de verdad se puede explotar (ver 7.5).
Objetivo específico 4: Analizar qué quiere decir que algunos controles quedaran en cumple parcial (ver 7.5).
Objetivo específico 5: Argumentar una respuesta técnica a la objeción del auditor externo (ver 7.6).

6. Justificación

Hacer esta evaluación sobre Compra Segura tiene sentido porque no es una app de ejemplo cualquiera, es una tienda que maneja usuarios, contraseñas y pedidos con precios, o sea que lo que guarda es justo lo que un atacante busca, cuentas y plata. En las tareas 1 y 2 ya habíamos encontrado fallas sueltas como la inyección en login.php, las claves en texto plano o el precio que viaja en un input hidden, pero una lista de fallas no dice qué tan lejos está el sistema de estar bien, entonces necesitábamos una forma de medirlo.
Usamos la ISO/IEC 27002 porque no es una lista de ataques sino de controles, entonces no solo pregunta si algo se puede romper sino si existe lo que debería existir, como los logs, los roles o un procedimiento para cuando pase algo, y eso nos dejó ver problemas que no son de código, como que no haya por dónde reportar un incidente. Además es la norma que se trabajó en clase, entonces lo

que sale de acá sirve de base para decidir qué corregir primero en el proyecto.

7. Cuerpo principal / Desarrollo

7.1 Contexto y metodología de evaluación

La evaluación se hizo sobre Compra Segura corriendo en local, en http://localhost/compraseguras/, con el mismo repositorio de las clases anteriores (ver Figura 1) y con el HALLAZGOS.md de la carpeta docs que veníamos llenando desde la clase 2. El código lo revisamos archivo por archivo (login.php, registro.php, pedido.php, buscar_producto.php y conexion.php) y la base de datos la revisamos desde phpMyAdmin.

Para cada control usamos el mismo procedimiento de cuatro pasos que se vio en clase:

Paso 1, identificar el control: o sea qué pide la norma, por ejemplo cifrado de la información sensible.

Paso 2, formular una pregunta verificable: una pregunta que se pueda responder con sí o no mirando el sistema, por ejemplo ¿la columna clave guarda hashes?

Paso 3, buscar la evidencia: abrir el archivo, buscar la función en todo el proyecto, mirar la tabla en phpMyAdmin o hacer la prueba directo en el navegador.

Paso 4, registrar el estado: Cumple, No cumple o Cumple parcial, y siempre con la evidencia al lado, porque sin eso es solo una opinión.

Así evaluamos los cinco dominios, A.9 Control de acceso, A.10 Criptografía, A.12 Seguridad de las operaciones, A.14 Adquisición, desarrollo y mantenimiento de sistemas y A.16 Gestión de incidentes, y en la 7.3 dejamos juntas todas las evidencias para que cada estado tenga de dónde agarrarse.

7.2 Evaluación de los dominios ISO 27002

7.2.1 A.9 Control de acceso

Abran registro.php y login.php. ¿Hay algún requisito mínimo de contraseña al registrarse? ¿Todos los usuarios tienen el mismo nivel de acceso, o existen roles diferenciados?

a. Para cumplir con el requisito de exigencia de la complejidad y longitud mínima en la creación de credenciales del sistema, primero nos preguntamos si hay algún requisito mínimo de contraseña al registrarse, y la respuesta es que no: primero, porque el sistema no tiene redireccionamiento; al lanzarse la app http://localhost/compraseguras/ 1. No se redirige a index.php, ni tiene htaccess. 2. No existe forma que el usuario llegue a registro.php, a no ser que sea escribiendo la url. 3. Si el usuario llega a registro.php, no tiene caracteres mínimos para la creación de usuarios. **(No cumple)**

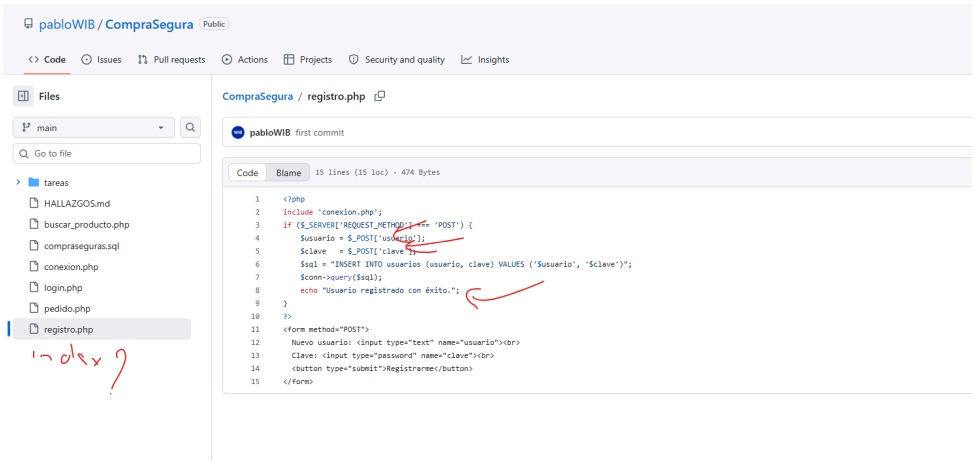


Figura 2. En esta imagen vemos la ausencia de index o un enrutamiento, y vemos registro.php: no tiene caracteres mínimos para el registro.

b. ¿El sistema valida que las contraseñas nuevas cumplan con un mínimo de caracteres y su complejidad en el archivo registro.php? No lo hace: en ninguna parte de las 15 líneas existe alguna instrucción de validación, ni por medio de funciones (js), porque no hay js, ni por php, y tampoco por html; esto lo probamos con un registro con usuario test y clave 1, y nos devuelve “Usuario registrado con éxito.”, y la fila queda creada en la tabla. **(No cumple)**

Code Blame 15 lines (15 loc) · 474 Bytes

```php
<?php
include 'conexion.php';
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $usuario = $_POST['usuario'];
    $clave   = $_POST['clave'];
    $sql = "INSERT INTO usuarios (usuario, clave) VALUES ('$usuario', '$clave')";
    $conn->query($sql);
    echo "Usuario registrado con éxito.";
}
?>
<form method="POST">
  Nuevo usuario: <input type="text" name="usuario"><br>
  Clave: <input type="password" name="clave"><br>
  <button type="submit">Registrarme</button>
</form>
```

Figura 3. En esta imagen vemos que el sistema no valida que las contraseñas nuevas cumplan con un mínimo de caracteres: solo permite el ingreso.

c. ¿Existen roles o niveles de acceso distintos? No: la tabla de usuarios (compraseguras.sql, líneas 4 a 8) tiene solo tres columnas, que son id, usuario y clave; no existe columna rol ni perfil o nivel, y así todos los usuarios comparten el mismo nivel de acceso. **(No cumple)**

Code Blame 33 lines (28 loc) · 831 Bytes

```sql
CREATE DATABASE compraseguras;
USE compraseguras;

CREATE TABLE usuarios (
    id INT AUTO_INCREMENT PRIMARY KEY,
    usuario VARCHAR(50),
    clave VARCHAR(50)
);

INSERT INTO usuarios (usuario, clave) VALUES
('admin', 'admin123'),
('jperez', 'miclave2024');
```

Figura 4. En esta imagen vemos solo usuarios y clave.

d. ¿Cada archivo verifica que exista sesión iniciada? (Después de preguntarle a Claude cómo funcionan las peticiones y que

auditara el login.php, mi respuesta). No, no se verifica; en sí, ahora el sistema de auth es solo decorativo, ya que los archivos no verifican si un usuario ya pasó por el login para permitirle hacer peticiones; entonces cualquiera puede saltárselos entrando directo a los archivos que hacen el trabajo real. La lógica de un sistema debería ser que, si no hay sesión, entonces cortar la ejecución ahí mismo y mandar al usuario a login; pero ahora no hay nada que lo verifique. **(No cumple)**

e. ¿La aplicación se conecta a la BD con una cuenta de mínimo privilegio? La aplicación usa la cuenta root del gestor de base de datos, como si en ningún momento se hubiera hecho ninguna configuración; también nos damos cuenta porque no hay ninguna llave guardada en un .env, y todo está guardado en los archivos sql, los cuales no están ignorados, por lo cual quien tenga acceso al repo, directamente también tiene acceso a la base de datos. **(No cumple)**

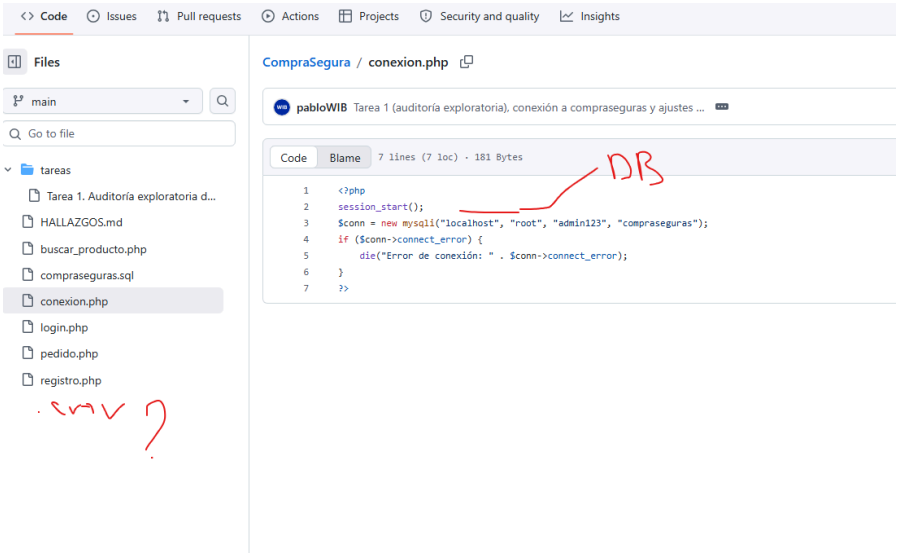


Figura 5. En esta imagen vemos cómo es la conexión directa, sin llaves, a la base de datos, y la ausencia de .env.

7.2.2 A.10 Criptografía

1. Revisen la tabla usuarios en phpMyAdmin, columna clave. Acá el objetivo es asegurar el uso apropiado de la criptografía para proteger la confidencialidad, la autenticidad e integridad de la información.

a. ¿La columna clave de la tabla usuarios almacena hashes? En este punto auditamos phpMyAdmin para revisar cómo están las tablas; por medio de una select revisamos la tabla de usuarios y podemos ver que, para cada usuario, se guarda id, nombre y clave (Sin cifrar); por ejemplo: “1 | admin | admin123 | 8”. **(No cumple)**

b. ¿El esquema soportaría un hash moderno? No lo soporta, ya que el sistema nunca fue diseñado para esto: primero, no tiene planeación; y, aunque se hagan medidas preventivas en el sistema, no es posible, ya que un hash ocupa 60 caracteres. **(No cumple)**

c. ¿Se invoca alguna función de hash o cifrado? Para esta parte revisamos los archivos php y encontramos 0 resultados, donde en ninguna parte del código se ve alguna función hash o verify; y, aunque esté mal nombrada, tampoco hay ningún patrón de función que cumpla con cifrados. **(No cumple)**

d. ¿El envío de credenciales viaja cifrado? No, y en este caso mucho menos; ya teniendo en cuenta cómo se ha fundamentado el sistema, esta parte no tiene como existir sin bases primarias. Pero, de igual forma, revisamos cómo se hacen las peticiones y vemos que se publica por POST sobre http://localhost/ sin ninguna configuración de HTTPS: las credenciales viajan directo y son legibles. **(No cumple)**

e. ¿Los secretos están fuera del código fuente? La contraseña del gestor de base de datos está escrita literalmente en conexion.php (línea 3) y publicada en el repo; tampoco existe archivo .env ni variables de entorno, ni .gitignore. **(No cumple)**

7.2.3 A.12 Seguridad de las operaciones

a. ¿Algún archivo registra quién inició sesión? Revisamos en login.php (líneas 13 a 15) y podemos ver que el sistema únicamente asigna variables de sesión e imprime "Bienvenido", pero no escribe en ningún archivo ni tabla. **(No cumple)**

b. ¿Se registran los intentos fallidos? Para este punto revisamos en login.php (líneas 11 a 18) y vemos que no hay ningún contador, ni marca de tiempo o bloqueo por intentos, ni registro de la IP de origen. **(No cumple)**

c. ¿Se usa alguna función de escritura de log? Buscamos en todos los php las funciones con las que normalmente se escribe un log, que son error_log, fopen y logger, y salieron 0 resultados; tampoco hay una carpeta logs ni ningún archivo .log en el repositorio: no existe. **(No cumple)**

d. ¿Se audita la creación de usuarios o de pedidos? No: registro.php (línea 6) mete el usuario nuevo y no deja constancia de quién lo creó, ni a qué hora, ni desde dónde; lo único que tiene fecha en todo el sistema es la columna fecha de la tabla pedidos, que se llena sola con CURRENT_TIMESTAMP. **(No cumple)**

e. ¿Los registros están protegidos contra alteración? De esta podemos decir que no alcanza ni a evaluarse, porque no hay registros que proteger; y, de todas formas, como la app se conecta con root, si alguien logra una inyección puede borrar o cambiar cualquier registro que lleguemos a tener más adelante. **(No cumple)**

f. ¿Los errores se registran en lugar de mostrarse al usuario? Esto incluso es al revés, porque en conexion.php (líneas 4 y 5), si la conexión falla, se hace un die imprimiendo el error que devuelve el gestor, y eso sale directo en el navegador del que esté visitando la página. Ahí se alcanza a ver información del servidor y de la base que nadie de afuera tendría por qué estar viendo. **(No cumple)**

7.2.4 A.14 Adquisición, desarrollo y mantenimiento de sistemas

a. ¿El código valida lo que el usuario envía antes de usarlo? No, porque pedido.php (líneas 4 a 6) agarra producto id, precio y cantidad del POST y los usa de una, directamente, sin mirar si son números, ni si están en un rango que tenga sentido, ni si son negativos; buscar_producto.php agarra lo que venga por GET. Pero buscamos en todo el proyecto las funciones con las que normalmente se valida, filter y match, y salieron 0 resultados. Esto lo probamos mandando cantidad -5, y el total sale en negativo y el pedido igual se guarda. **(No cumple)**

b. ¿Las consultas separan código de datos? No: las cuatro consultas que tiene el sistema (login.php línea 8, registro.php línea 6, buscar_producto.php línea 5 y pedido.php línea 10) arman el sql

pegando directamente lo que escribe el usuario. Está en el login, en el registro, en la búsqueda y en el pedido, y en los cuatro es lo mismo. Aquí, para comprobarlo, mandamos una comilla sola en la búsqueda, buscar_producto.php?q=' y el gestor se rompe y tira el error; confirmamos que el parámetro sí se puede inyectar. **(No cumple)**

c. ¿El precio de la transacción se determina en el servidor? No, y esta nos pareció de las más graves: en pedido.php (línea 17) el precio va en un input hidden con el valor 3000, y el servidor lo recibe y lo multiplica por la cantidad sin ir nunca a la tabla productos a ver cuánto vale ese producto de verdad. Le cambiamos el hidden a 1 desde el navegador, y el sistema respondió: "Pedido registrado. Total a pagar: $1" por el mismo producto de 3000 (supuestamente). **(No cumple)**

d. ¿Se escapa lo que se imprime al navegador? No, porque login.php imprime el nombre del usuario directo en el mensaje de bienvenida (línea 15), y buscar_producto.php imprime el nombre del producto igual (línea 8); y, como en el registro tampoco se filtra lo que entra, entonces alguien puede registrarse con un nombre que tenga etiquetas html o js, y eso queda guardado en la base y se ejecuta cada vez que se muestra. **(No cumple)**

e. ¿Los formularios incluyen token de un solo uso? Ninguno de los cuatro formularios lo tiene: ni el login, ni el registro, ni el de pedido, ni el de búsqueda; y tampoco hay en el código nada que genere o verifique un token, o sea, ese mecanismo no existe. **(No cumple)**

f. ¿La base de datos impone restricciones? No: la tabla pedidos declara usuario id y producto id como INT normales (compraseguras.sql, líneas 28 y 29), sin FOREIGN KEY y sin NOT NULL; entonces así se pueden guardar pedidos que apuntan a usuarios y productos que no existen. **(No cumple)**

7.2.5 A.16 Gestión de incidentes de seguridad de la información

a. ¿Existe un procedimiento escrito de respuesta a incidentes? No hay nada: no existe un SECURITY.md, ni una política, ni un plan escrito en ninguna parte del repo. **(No cumple)**

b. Si un cliente reporta hoy un problema, ¿por dónde lo haría? No hay forma de hacerlo: la aplicación no publica correo de contacto, formulario de reporte ni aviso legal. **(No cumple)**

c. ¿Existe un mecanismo para registrar debilidades detectadas? No desde un comienzo; pero, teniendo en cuenta los trabajos, creé docs/ con archivos md, HALLAZGOS y otros, para ir documentando el proyecto y sus problemas. **(Cumple parcial)**

d. ¿Los eventos se clasifican por severidad? El archivo HALLAZGOS.md tiene una columna de gravedad con escala, que está aplicada a los hallazgos; pero la escala no está definida formalmente, y nunca staging y menos prod. **(Cumple parcial)**

e. ¿Se puede investigar un incidente? Al no existir ningún registro de eventos, queda imposible recolectar evidencia: no se podría determinar si ocurrió, ni cuándo, ni qué se accedió. **(No cumple)**

f. ¿Los hallazgos alimentan mejoras? Sí, pero en parte: teniendo en cuenta que el sistema no tiene planeación, lo que se ha documentado es solo informativo, pero no se ha puesto en práctica donde se van priorizando las correcciones. Es un ejercicio real de lecciones que vamos aprendiendo, pero no se ha traducido en ninguna corrección en el código. **(Cumple parcial)**

7.2.6 Resumen del checklist

Esta tabla junta en un solo lugar lo que evaluamos en la 7.2, con el estado de cada dominio y la evidencia con el archivo y la línea donde se ve.

Dominio	Objetivo de control	Control específico	Estado	Evidencia concreta
A.9 Control de acceso	Gestión de acceso de usuarios	Contraseñas seguras y gestión de privilegios	No cumple	registro.php (líneas 4 a 6) guarda la clave sin validar nada, probado con clave 1; la tabla usuarios no tiene

				columna de rol (compraseguras.sql, líneas 4 a 8); ningún archivo revisa la sesión antes de trabajar; conexion.php (línea 3) se conecta con root.
A.10 Criptografía	Controles criptográficos	Cifrado de información sensible	No cumple	Claves en texto plano en compraseguras.sql (líneas 10 a 12) y columna clave VARCHAR(50) que no alcanza para un hash de 60 caracteres (línea 7); no hay password_hash en ningún php; la contraseña de la base está escrita en conexion.php (línea 3); el login viaja por http.
A.12 Seguridad de las	Registro y seguimiento	Registro de eventos (logs)	No cumple	login.php (líneas 11 a 18) solo

operaciones				guarda la sesión e imprime el mensaje, no registra quién entró ni los intentos fallidos; 0 resultados buscando error_log, fopen y logger; conexion.php (líneas 4 y 5) muestra el error en pantalla con die en vez de registrarlo.
A.14 Adquisición, desarrollo y mantenimiento	Seguridad en el desarrollo	Validación de datos de entrada	No cumple	pedido.php (líneas 4 a 8) usa producto, precio y cantidad del POST sin validar y el precio viaja en un hidden (línea 17); buscar_producto.php toma q del GET (línea 3) y lo pega en el SQL (línea 5); lo mismo en login.php (línea 8) y registro.php (línea 6).
A.16	Gestión de	Procedimie	No cumple	No hay

Gestión de incidentes	incidentes y mejoras	nto de respuesta a incidentes	(3 puntos en cumple parcial)	SECURITY.md, política ni canal de reporte en el repo ni en la app; lo único que existe es HALLAZGOS.md con una columna de gravedad, hecho a mano durante la clase.

7.3 Evidencias técnicas

Acá dejamos juntas las evidencias que usamos en la 7.2, para que cada estado tenga de dónde agarrarse. Las que tienen captura están como figuras dentro de la 7.2, y las otras son pruebas que hicimos directo en el navegador o en phpMyAdmin y que cualquiera puede repetir.

Evidencias con captura:

Figura 1, repositorio del proyecto: es el código que auditamos, el mismo de las clases anteriores.

Figura 2, registro.php y la raíz de la app: no hay index ni enrutamiento, a registro.php solo se llega escribiendo la url y no hay mínimo de caracteres para la clave (A.9).

Figura 3, prueba de registro con usuario test y clave 1: el sistema responde “Usuario registrado con éxito.” y crea la fila, o sea que no valida longitud ni complejidad (A.9).

Figura 4, tabla usuarios en phpMyAdmin: solo tiene id, usuario y clave, no hay columna de rol (A.9).

Figura 5, conexion.php (línea 3) y la raíz del repo: la conexión es con root, la contraseña está escrita en el código y no hay .env (A.9 y A.10).

Figura 6, formulario de login.php: con ' OR ''=' en usuario y clave entra como admin sin contraseña (7.4).

Pruebas sin captura:

Select a la tabla usuarios en phpMyAdmin (compraseguras.sql, líneas 10 a 12): la clave de admin se ve tal cual, admin123, en texto plano (A.10).
Búsqueda de error_log, fopen y logger en todos los php: 0 resultados, no hay logs (A.12).
Cantidad -5 en pedido.php (línea 8, donde el total es precio por cantidad): el total sale negativo y el pedido igual se guarda (A.14).
buscar_producto.php?q=' en la búsqueda (línea 5): el gestor se rompe y muestra el error, entonces el parámetro se puede inyectar (A.14).
Hidden del precio (pedido.php, línea 17) cambiado a 1 desde el navegador: el sistema responde “Pedido registrado. Total a pagar: $1” (A.14).
Revisión del repo y de la app buscando un SECURITY.md, una política o un canal de reporte: no hay nada (A.16).

7.4 Hallazgo de autenticidad

a.

Code Blame 25 lines (22 loc) · 766 Bytes

```php
<?php
include 'conexion.php';

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $usuario = $_POST['usuario'];
    $clave   = $_POST['clave'];

    $sql = "SELECT * FROM usuarios WHERE usuario = '$usuario' AND clave = '$clave'";
    $resultado = $conn->query($sql);

    if ($resultado && $resultado->num_rows > 0) {
        $fila = $resultado->fetch_assoc();
        $_SESSION['usuario_id'] = $fila['id'];
        $_SESSION['usuario'] = $fila['usuario'];
        echo "Bienvenido, " . $fila['usuario'] . ". Acceso concedido.";
    } else {
        echo "Usuario o clave incorrectos.";
    }
}
?>
<form method="POST">
  Usuario: <input type="text" name="usuario"><br>
  Clave: <input type="password" name="clave"><br>
  <button type="submit">Ingresar</button>
</form>
```

Figura 6. Formulario de acceso de login.php, donde la cadena ' OR ''=' da acceso como admin sin contraseña.

El formulario de acceso recibe usuario / clave y los inserta, sin ningún tratamiento, dentro de una cadena sql (login.php, línea 8).

Resultado: cualquier persona en Internet puede iniciar sesión como `admin` sin credenciales.

Prueba ejecutada (bypass de autenticación)

Usuario: `' OR ''='`
Clave: `' OR ''='`

1. Concede el acceso sin conocer ninguna contraseña.

2. Asigna a la sesión la identidad de la primera fila de la tabla, que, según compraseguras.sql, es admin.
3. Responde: *"Bienvenido, admin. Acceso concedido."*

7.4.1 Situación observada

Lo que le falta al login es una verificación real de identidad. En login.php (líneas 5, 6 y 8) el usuario y la clave que llegan del formulario se pegan directo dentro de la consulta, entonces el sistema no compara la clave de nadie, solo le pregunta a la base si la consulta devuelve alguna fila. Con ' OR ''=' la condición siempre da verdadero, la base devuelve la primera fila, que es admin, y el sistema guarda ese id y ese nombre en la sesión (líneas 13 y 14) como si esa persona se hubiera autenticado. No hay consulta preparada, no se compara ningún hash con password_verify, no hay límite de intentos ni un segundo factor, o sea que no existe nada que garantice que quien entra es de verdad el dueño de la cuenta.

7.4.2 ¿Por qué compromete la autenticidad y no otro principio?

Porque lo que se rompe acá es la garantía de que el que entra es quien dice ser. La autenticación es el proceso, o sea pedir usuario y clave, y la autenticidad es el resultado, que el sistema pueda confiar en esa identidad. En este caso el proceso sí existe, porque el formulario pide credenciales, pero el resultado es falso, porque el sistema da por hecho que somos admin sin haberlo comprobado.
Y es un fallo distinto al de confidencialidad, aunque venga de la misma inyección. La confidencialidad se rompe cuando alguien ve información que no debería ver, como las claves en texto plano de la tabla usuarios, y eso puede pasar sin suplantar a nadie. Acá es al revés:

aunque el atacante no leyera ningún dato, igual queda registrado como admin, y todo lo que haga después, como un pedido, queda a nombre de admin (pedido.php, línea 7). Y tampoco toca la integridad ni la disponibilidad, porque en la prueba no modificamos ningún dato ni tumbamos el servicio, solo entramos con una identidad que no era la nuestra.

7.5 Discusión final

1. **¿Si tuvieran que corregir un solo dominio primero, cuál sería y por qué?**

   Corregiríamos primero adquisición y mantenimiento de sistemas, y dentro de ese dominio, puntualmente, lo de separar código de datos en las cuatro consultas.

   Para esto tenemos tres razones. La primera es que es el único que tiene hallazgos que se pueden explotar desde afuera sin tener nada: ni cuenta, ni sesión, ni acceso a la red; el bypass del login se hace escribiendo una cadena en un campo de un formulario que está público. Los otros hallazgos son graves, pero necesitan una condición antes; por ejemplo, para aprovechar que las claves están en texto plano, primero hay que llegar a la base de datos, y la forma más fácil de llegar a la base de datos es, justamente, la inyección.

   La segunda es que es la corrección que más cubre con menos trabajo: pasar las cuatro consultas a sentencias preparadas es un cambio, y cierra de una sola vez el bypass del login, la inyección de buscar_producto.php, la del registro y la de pedido.php; o sea, un solo tipo de cambio cierra cuatro puertas.

   Y la tercera, que es la que más nos pesó, es que mientras la inyección siga ahí, los otros controles no se sostienen, aunque los implementemos: si mañana hacemos los logs de operaciones, pero la app sigue conectándose con root y sigue siendo inyectable, el atacante puede borrar los mismos logs, y el control queda existiendo, pero sin servir; y si implementamos roles, pero el atacante entra directo como admin saltándose el login, el control de roles tampoco sirve. Por eso este dominio va de primero: porque es la base sobre la que los otros se vuelven creíbles.

2. ¿Qué significan los hallazgos que quedaron en cumplimiento parcial?

Los tres cumple parcial nos quedaron todos en gestión de incidentes, en el mecanismo para registrar debilidades, en la clasificación por severidad y en que los hallazgos alimenten mejoras; y quedaron ahí por una razón concreta, y es que esas tres cosas sí están pasando, pero no las está haciendo el sistema, porque las estamos haciendo nosotros como ejercicio de clase. O sea: sí hay un registro de debilidades, pero es la carpeta docs con el HALLAZGOS.md que creamos nosotros; sí hay una clasificación por severidad, pero es una columna de gravedad que llenamos a mano y cuya escala nunca quedó definida; y sí hay hallazgos que van a alimentar mejoras, pero hasta ahora es informativo, porque no se ha traducido en una sola corrección dentro del código.

Nada de eso vive dentro de la aplicación, ni existiría si nosotros no estuviéramos haciendo la auditoría. Eso es lo que significa un cumplimiento parcial.

Lo que sacamos de ahí es que un cumple parcial no es medio control: es un control que todavía no existe, pero que por ahora alguien está supliendo a mano; y, para volverlo un cumple, toca escribir el procedimiento, decir quién responde, decir en cuánto tiempo se responde y dejar el registro dentro del sistema, y no dentro de un trabajo de clase.

7.6 Reto del «Auditor Externo»

El auditor externo nos dice que marcamos A.16 como no cumple, pero que eso en realidad es un problema de A.9, porque si Compra Segura tuviera buenos controles de acceso no habría incidentes que gestionar.
Nuestra respuesta es que no, porque los dos dominios tienen objetivos de control distintos. A.9 busca que solo entre quien debe entrar y que cada uno haga solo lo que le toca, o sea que su trabajo es prevenir. A.16 busca que, cuando algo pase, exista cómo detectarlo, reportarlo, responder y aprender de eso, o sea que su trabajo es reaccionar. Un control que previene no reemplaza a uno que responde, porque ningún control de acceso deja el riesgo en cero, y por eso la norma los tiene separados.

Y aunque A.9 quedara perfecto en Compra Segura, con contraseñas seguras, roles y la sesión validada en cada archivo, igual podrían pasar incidentes por otras causas:
Errores humanos: el mismo admin puede borrar productos por equivocación, o alguien puede subir al repo un archivo con la contraseña de la base, que es lo que ya pasa hoy con conexion.php (línea 3), y eso no tiene nada que ver con quién inicia sesión.
Usuarios con permisos legítimos: un empleado que sí tiene acceso puede cambiar precios o sacar datos de los clientes, y para A.9 eso es un acceso correcto, porque entró con su cuenta y tenía el permiso.
Fallas de lógica que no pasan por el login: el precio del input hidden (pedido.php, línea 17) se puede cambiar estando logueado de forma normal, entonces el control de acceso lo deja pasar igual.
Problemas de disponibilidad: si se cae el servidor o el gestor de base de datos, o se daña una tabla, nadie atacó nada, pero igual es un incidente, y hoy ni siquiera quedaría registrado, porque conexion.php (líneas 4 y 5) solo muestra el error en pantalla.
En todos esos casos A.9 no sirve de nada, y lo que hace falta es justo lo que evalúa A.16: un procedimiento escrito, un canal para que el cliente reporte, una forma de clasificar por gravedad y registros para poder investigar. Por eso mantenemos A.16 como un dominio aparte y en no cumple, aunque esté relacionado con A.9: mejorar el acceso baja la cantidad de incidentes, pero no dice qué hacer cuando igual pasan.

8. Discusión general

Cuando ponemos todos los hallazgos juntos se ve que no son fallas sueltas, sino que se van encadenando entre sí. La inyección de login.php deja entrar como admin sin clave, y como la app se conecta con root esa misma inyección llega a toda la base; ahí están las claves en texto plano, entonces el atacante sale con las contraseñas reales de todos, y como no hay ni un log nadie se entera, y aunque alguien se diera cuenta tampoco hay procedimiento ni canal para responder. O sea que A.14, A.9, A.10, A.12 y A.16 terminan siendo eslabones de la misma cadena, y cada uno que falla hace peor al siguiente.

Por eso priorizamos por impacto real y no por cuántas fallas tiene cada dominio. De primeras van la inyección y el precio en el hidden, porque se explotan desde afuera sin nada, solo con el navegador, y la del precio ya es plata directa para el negocio. Después van las claves en texto plano y la conexión con root, porque necesitan que alguien llegue a la base primero, pero cuando llega el daño es total. Y más abajo quedan los logs y la gestión de incidentes, que no dejan entrar a nadie,

pero son los que deciden si nos damos cuenta y si podemos investigar lo que pasó.
Una objeción que nos podrían hacer es que marcamos casi todo en no cumple y que eso es exagerar con una app de clase. Pero cada no cumple tiene su archivo, su tabla o su prueba al lado (ver 7.3), y donde encontramos algo que sí existía lo marcamos, que son los tres cumple parcial de A.16. O sea que el resultado no sale de querer que todo falle, sale de que el sistema nunca se pensó con seguridad desde el principio, y eso mismo muestran los tres cumple parcial: lo poco que hay lo estamos poniendo nosotros desde afuera y no la aplicación.

9. Conclusiones

a. (Objetivo específico 1) La aplicación Compra Segura presenta un nivel de cumplimiento muy bajo frente a los cinco dominios evaluados del estándar ISO/IEC 27002 (Control de acceso, Criptografía, Seguridad de las operaciones, Adquisición y mantenimiento de sistemas, y Gestión de incidentes), con la gran mayoría de los controles revisados en estado “No cumple” y solo tres controles en “Cumple parcial”, todos ellos sostenidos por la documentación creada durante el ejercicio de auditoría y no por la aplicación en sí misma.
b. (Objetivo específico 2) El hallazgo más grave corresponde al principio de autenticidad en el formulario de inicio de sesión: al no separar código de datos en la consulta SQL, cualquier persona en internet puede iniciar sesión como admin sin conocer ninguna credencial, usando la cadena ' OR ''='. Este hallazgo cumple con el objetivo central de la actividad, ya que quedó documentado y justificado con evidencia reproducible (prueba ejecutada, entrada utilizada y resultado obtenido).

c. (Objetivo específico 3) En respuesta a la discusión final, se determinó que el dominio a corregir con mayor prioridad es Adquisición, desarrollo y mantenimiento de sistemas, específicamente la separación de código y datos en las cuatro consultas SQL del sistema, dado que es el único hallazgo explotable sin requisitos previos, es el que cierra la mayor cantidad de vulnerabilidades con el menor esfuerzo, y es la base que permite que los demás controles (roles, cifrado, logs) tengan sentido una vez implementados.

d. (Objetivo específico 4) Los tres hallazgos en “Cumple parcial” (registro de debilidades, clasificación por severidad y aprovechamiento de hallazgos para mejoras) evidencian que la gestión de incidentes que existe hoy depende por completo del trabajo manual del equipo auditor durante la clase, y no de un proceso institucionalizado dentro del sistema; por lo tanto, no deben interpretarse como un avance real de la aplicación, sino como una oportunidad de formalización pendiente.

e. (Objetivo específico 5) Frente a la objeción del auditor externo, concluimos que A.16 no se puede meter dentro de A.9, porque A.9 previene que entre quien no debe y A.16 responde cuando algo igual pasa; aun con un control de acceso perfecto seguirían pasando incidentes por errores humanos, por usuarios con permisos legítimos, por fallas de lógica como el precio del input hidden o por caídas del servicio, entonces los dos dominios se evalúan por separado y A.16 se mantiene en no cumple.
f. (Objetivo general) Con todo lo anterior se cumple lo que nos propusimos al inicio, porque quedaron evaluados los cinco dominios con su estado y su evidencia, el hallazgo de autenticidad quedó documentado con una prueba que se puede repetir, respondimos las dos preguntas de la discusión y le dimos respuesta a la objeción del auditor externo; y lo que nos queda claro es que antes de agregarle cosas nuevas a Compra Segura toca corregir la base, empezando por las consultas.

10. Bibliografía

International Organization for Standardization. (2022). ISO/IEC 27002:2022. Information security, cybersecurity and privacy protection — Information security controls. ISO.

The PHP Group. (s. f.). Sentencias preparadas y procedimientos almacenados. Manual de PHP. https://www.php.net/manual/es/pdo.prepared-statements.php

Manual de PHP. https://www.php.net/manual/es/function.password-hash.php

11. Anexos

Anexo A. Interacción con herramientas de inteligencia artificial

En este trabajo usé Claude como apoyo durante el desarrollo de la actividad, actuando como auditor, no tanto en la parte de lógica y técnica sino más que todo en la presentación y entrega del proyecto, creación de tablas, agregando información que yo ya tenía en el documento, agregar signos de puntuación, en todo el documento, creación de índice y portada

Le pregunté cómo funcionan las peticiones entre los archivos php y le pedí que auditara login.php.

Resumen de la respuesta: que login.php solo asigna variables de sesión, pero los otros archivos no revisan si esa sesión existe, entonces se puede entrar directo a ellos sin pasar por el login.

Cómo lo verifiqué: lo contrasté revisando los archivos php del proyecto, y lo que quedó escrito en la pregunta de sesión iniciada (7.2.1) es mi respuesta después de revisarlo.